

RAGFlow 企业级 RAG 引擎部署指南

文档版本：v2.5 | 发布日期：2026年4月 | 适用版本：RAGFlow 0.25.1+

1. 系统概述

RAGFlow 是一款基于深度文档理解的开源 RAG（Retrieval-Augmented Generation）引擎。它通过深度文档解析、智能分块和多路检索技术，为企业提供高精度的知识库问答能力。本指南涵盖了从环境准备到生产部署的完整流程。

2. 环境要求

- 2.1 操作系统：Ubuntu 22.04 LTS / CentOS 8+ / Debian 12
- 2.2 Python 版本：3.11.0 及以上
- 2.3 CPU 最低要求：8 核 | 内存最低要求：32 GB | 磁盘：SSD 500 GB 以上
- 2.4 GPU（可选）：NVIDIA T4 / A10 / A100 用于 DeepDoc OCR 和本地 LLM 推理
- 2.5 依赖服务：MySQL 8.0、Elasticsearch 8.11+、Redis 7.2+、Milvus 2.4+、MinIO

3. 快速安装

3.1 Docker Compose 安装（推荐）

- 步骤1：克隆代码仓库
git clone https://github.com/infiniflow/ragflow.git && cd ragflow
- 步骤2：配置环境变量
cp .env.example .env # 编辑配置 MySQL 密码、ES 地址、LLM API Key 等
- 步骤3：启动服务
docker compose -f docker/docker-compose.yml up -d
- 步骤4：验证安装 — 访问 http://localhost:9380 查看 RAGFlow Web 界面

3.2 Kubernetes 部署

- kubectl create namespace ragflow
- helm install mysql bitnami/mysql -n ragflow --set auth.rootPassword=ragflow123
- helm install elasticsearch elastic/elasticsearch -n ragflow --set replicas=3
- kubectl apply -f k8s/ragflow-deployment.yaml -n ragflow
- kubectl apply -f k8s/ragflow-service.yaml -n ragflow

4. 核心配置说明

4.1 文档解析引擎配置

- RAGFlow 内置 DeepDoc 解析引擎，支持 PDF、DOCX、PPTX、XLSX、CSV、TXT、MD、HTML 等格式。
- parser.pdf.default: deepdoc # 备选: mineru, docling, paddleocr

4.2 分块策略配置

- RAGFlow 提供15种分块方法：General、QA、Paper、Laws、Book、Presentation、Manual、Table、Email、One、Tag、KnowledgeGraph、Mix、Audio、Medical。推荐根据文档类型选择最合适的方法。

分块大小建议：512 tokens（通用）、256 tokens（精确查询）、1024 tokens（长文理解）。重叠 Token 建议 50-100。

4.3 嵌入与LLM模型配置

推荐嵌入模型：BAAI/bge-large-zh-v1.5（中文最佳，1024维）、BAAI/bge-m3（多语言）、text-embedding-3-large（OpenAI）

推荐LLM模型：Qwen2.5-72B-Instruct、DeepSeek-V3、Llama-3.3-70B-Instruct、Claude-3.5-Sonnet

5. RAG 3.0 增强配置

RAGFlow	从	v0.25.0	支持	RAG	3.0
---------	---	---------	----	-----	-----

多通道检索架构，包括：分类器路由（4分类器：意图/复杂度/文档类型/安全分级）、PageIndex

树索引（无向量推理检索）、LLM Wiki 知识编译（三层编译：原始→实体→综合）、GraphRAG 知识图谱、RRF 融合 + Cross-Encoder 精排。

6. 安全与权限配置

访问控制：知识库级、文档级、Chunk级三级ACL

数据加密：传输层 TLS 1.3，存储层 AES-256

PII脱敏：身份证号、手机号、银行卡号、邮箱正则脱敏

审计日志：全链路操作审计，支持导出和合规报告

7. 性能优化

向量索引：Milvus 使用 IVF_FLAT 或 HNSW，nlist=sqrt(总向量数)

ES优化：每索引3-5分片，SSD存储

缓存：Redis 热门查询结果缓存 TTL=1h，LRU 淘汰策略

并行：Celery 分布式任务队列并行处理文档解析和索引构建

8. 监控与运维

健康检查：GET /api/v1/health 返回各组件状态

性能指标：Prometheus+Grafana 监控 QPS、P95延迟、错误率、GPU利用率

备份策略：数据库每日全量+每小时增量，Milvus 每日快照

9. 常见问题

Q：文档解析失败？A：检查加密/损坏；确认引擎配置；查看worker日志。

Q：检索不相关？A：调整分块策略；更换嵌入模型；启用Reranker精排。

Q：查询响应慢？A：检查Milvus索引；确认向量索引类型；分析Span耗时。